

O'Connors AI Platform — Comprehensive Master Design Document

Date: 2026-06-03

Authors: Capstone Project Team, University of Adelaide

Classification: Internal — O'Connors Capstone Project

1. Executive Summary & Project Overview

This document presents the comprehensive master design for the O'Connors AI Platform. It integrates the Concept Design, the LLM and Vector Database Solution Analysis, and the Detailed Service Design Document into a single, unified architectural specification.

The O'Connors AI Platform is an enterprise-grade Retrieval-Augmented Generation (RAG) chatbot system tailored for O'Connors, a leading HVAC and mechanical engineering firm based in Kent Town, South Australia. The platform enables O'Connors staff to query internal Quality, Safety, and Environment (IMS) documentation, technician reference materials, project files, and tendering records using natural language and receive cited, accurate answers in real time.

Core capabilities include:

- Secure single-sign-on via Microsoft Azure Active Directory (Entra ID), bridged to a platform JWT.
- Real-time WebSocket chat with streaming "typing" animation and persistent chat session history.
- Advanced RAG pipeline with hybrid BM25 + vector search, domain-filtered retrieval, and cross-encoder re-ranking.
- In-browser document preview (PDF, Word, Excel, images) linked directly to AI source citations.
- Scheduled ETL pipeline that ingests, OCRs, and indexes updated SharePoint documents every 3 days.

Compliance with the Australian Signals Directorate's **Essential Eight** framework is mandatory. The target deployment is Azure Container Apps (proudground-90080d26.australiaeast.azurecontainerapps.io), with a strict isolation policy where only the Kong API Gateway is publicly accessible.

2. Methodology & Microservice Architecture

2.1 Research & Methodology

The architecture is grounded in a structured research process combining academic literature, industry case studies (Netflix, Uber, Amazon, Spotify), and comparative technology evaluation. The functional requirements (natural-language querying, secure Azure AD auth, real-time chat, scheduled ingestion) drove the selection of a microservice architecture over a monolithic approach.

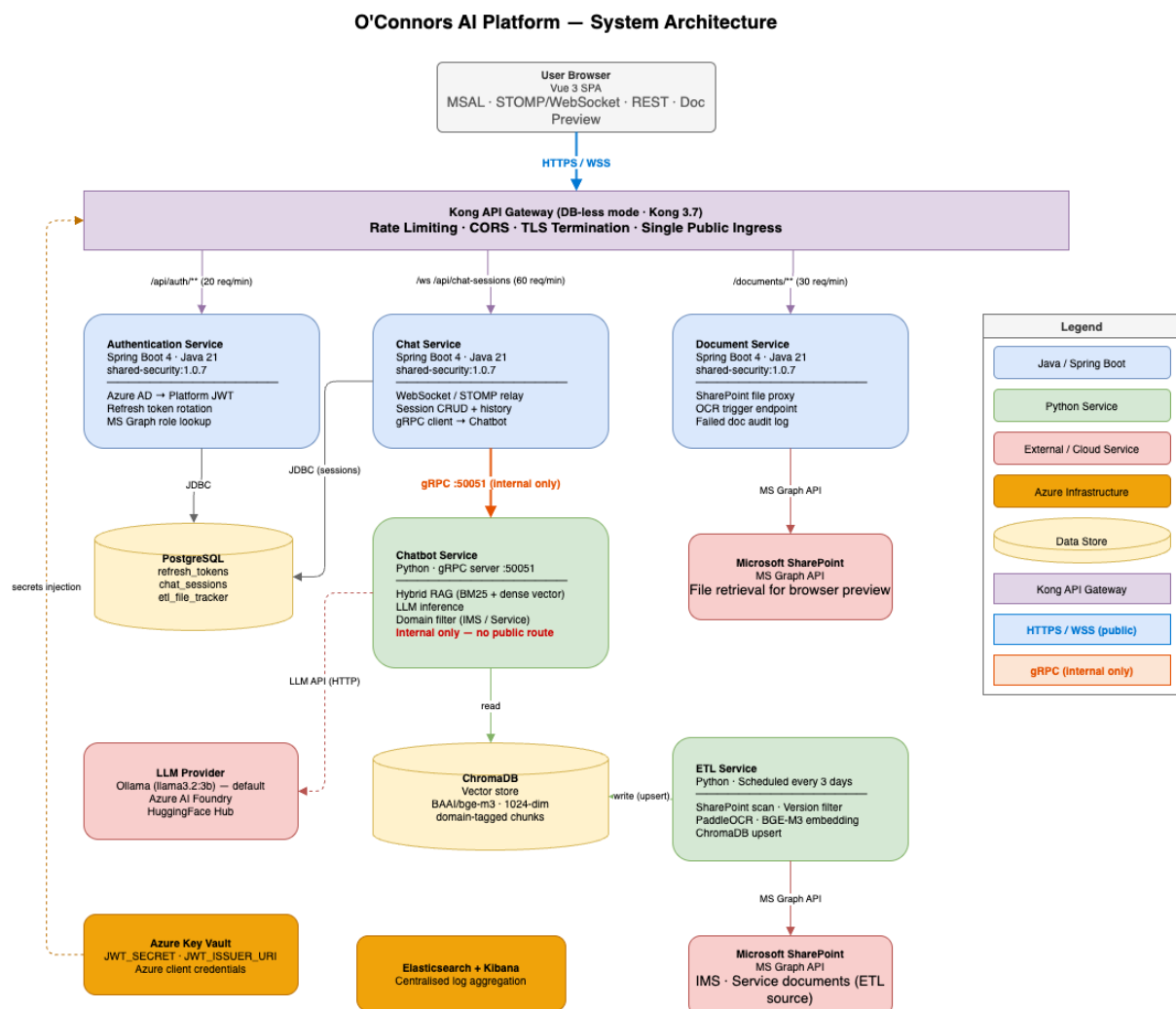
2.2 Why Microservices?

Microservices structure a system as a collection of small, independently deployable services, each responsible for a single bounded domain (Fowler & Lewis, 2014). This project adopts microservices for the following reasons:

- **Technology Heterogeneity:** Python is required for ML libraries (PyTorch, PaddleOCR), while Java is ideal for enterprise business services (Spring Boot). A monolith cannot easily embed these together.
- **Independent Scalability:** The AI inference engine requires high CPU/GPU bursts per query, while the Auth and Chat services are I/O bound. Azure Container Apps allows each to scale independently.
- **Fault Isolation:** As demonstrated by Netflix (Cockcroft, 2016), if the AI engine crashes, the chat service can return a graceful degradation response without bringing down the entire platform.
- **Team Autonomy:** Each service acts as a discrete functional domain with an independent CI/CD pipeline, enabling parallel development (Taibi & Lenarduzzi, 2018).

3. System Architecture & Data Flow

3.1 High-Level Architecture



3.2 Communication Protocol Design

The platform uses three distinct protocols tailored to specific communication patterns:

Communication Path	Protocol	Rationale
Browser → Kong (Auth, Docs)	HTTPS (REST)	Universal standard. Best for stateless, client-initiated request-response operations.
Browser → Kong (Chat)	WebSocket (STOMP)	Full-duplex persistent channel. Eliminates HTTP polling latency and bandwidth waste (Pimentel & Nickerson, 2012).
Chat Service → Chatbot Service	gRPC (HTTP/2)	High performance, binary serialisation (protobuf). Strongly typed contracts between Java and Python (Venkataraman et al., 2018).

4. Detailed Service Design

4.1 Kong API Gateway

Acts as the single public entry point (Kong 3.7, DB-less). It centralises JWT rate limiting and CORS. Configurations are templated at startup via `envsubst` to inject `JWT_SECRET` from Azure Key Vault.

4.2 Authentication Service

Tech: Java 21, Spring Boot 4.0.4, PostgreSQL. Bridges Azure AD OAuth2 with a platform-specific short-lived JWT + refresh token pair. Token rotation is enforced (old tokens revoked on use), and refresh tokens are stored as SHA-256 hashes in PostgreSQL.

4.3 Chat Service

Tech: Java 21, Spring Boot, STOMP. Manages chat sessions and WebSocket relays. Stores chat history as a JSON column in PostgreSQL to avoid relational join overhead. Relays messages via gRPC to the Chatbot Service.

4.4 Chatbot Service (RAG Engine)

Tech: Python, gRPC, ChromaDB, LangChain. Strictly internal (port 50051). Performs Retrieval-Augmented Generation using a hybrid BM25 + vector search strategy. Supports `ollama`, `huggingface`, or `azure-foundry` backends switchable via environment variables.

4.5 ETL Service

Tech: Python 3.12, LangChain, PaddleOCR. Runs every 3 days. Scans SharePoint, filters versions (keeping only the latest base document while preserving amendments), extracts text (falling back to PaddleOCR for scanned PDFs), chunks data with contextual enrichment, and embeds using `BAAI/bge-m3` into ChromaDB.

4.6 Document Service & User Interface

Document Service (Java 21): Proxies SharePoint to serve raw document bytes for in-browser preview securely.

User Interface (Vue 3/TypeScript): SPA providing chat interface with typing animations, document viewing, MSAL-based sign-in, and A/B testing response comparison capabilities.

5. AI Engine, Vector Database & RAG Strategy

5.1 Large Language Model Analysis

The O'Connors' Hyper-V environment supports quantised models in the 7B–14B parameter range comfortably.

- **Open-Source:** Meta's Llama 3.1 8B or Microsoft's Phi-4 14B served via Ollama offer zero recurring licensing costs.
- **Enterprise:** Azure OpenAI (GPT-4o or GPT-4o-mini) is the strongest cloud candidate. However, real-world Azure deployments incur 15–40% overhead, pushing costs to \$150–\$500/month for moderate usage, and fine-tuned hosting adds \$50–70/day.
- **Recommendation:** Use a hybrid approach—on-premise open-source models for routine queries to contain costs, with Azure OpenAI activated only for complex edge cases.

5.2 Vector Database Analysis

Cloud-managed vector databases (like Pinecone at \$70–\$300/month) are not cost-justified for O'Connors' scale (< 1 million vectors).

- **Recommendation:** ChromaDB is used for prototyping due to its simple API. For production, the system targets Qdrant or Weaviate, both of which are self-hosted, Docker-deployable, and completely free.

5.3 RAG Architecture (The 3 Tiers)

The RAG pipeline determines accuracy more than the LLM.

- **Tier 1 (Foundation):** Hybrid RAG (Vector + BM25) is the baseline. Essential for HVAC documentation mixing precise technical terms with general language.
- **Chunking & Enrichment:** 1,500–2,000 tokens per chunk. Contextual chunking prepends document-level metadata (Anthropic methodology) before embedding with [BAAI/bge-m3](#).
- **Retrieval Enhancement:** Cross-encoder re-ranking (retrieving top 50, re-scoring to top 5–10) is the highest-impact enhancement implemented.

6. Financial & Cost Analysis

Component	Open-Source (On-Prem)	Azure OpenAI (Cloud)	Hybrid (Selected)
LLM Inference	\$0 licence; ~\$2–4K GPU hardware (one-off)	\$2.50–10/M tokens; ~\$150–500/mo at moderate use	\$0 baseline + Azure overflow
Vector Database	\$0 (Qdrant/ChromaDB self-hosted Docker)	\$70–300/mo (Pinecone/Azure AI Search)	Self-hosted primary

Embedding Model	\$0 (BGE-m3 local)	\$0.02/M tokens (Azure ada-002)	Local default
Estimated Year 1 Total	\$2–4K (hardware only)	\$3–8K+ recurring	\$2–4K + minimal Azure fees

7. Data & Security Architecture

Database-per-Service Pattern: PostgreSQL houses isolated schemas for auth (refresh tokens) and chat (sessions, ETL tracking). ChromaDB handles vector storage.

Security Layers:

1. **Transport:** TLS 1.3 encryption across all public WSS/HTTPS traffic.
2. **Gateway:** Kong rate limiting and CORS enforcement.
3. **Identity:** Azure AD PKCE flow exchanged for a platform JWT (HS256). Refresh tokens are hashed via SHA-256 in the database.
4. **Authorisation:** Spring Boot validates the platform JWT and enforces role-based access control.
5. **Isolation:** Backend services set to `external: false`. Azure Key Vault injects secrets at runtime.

8. Test Cases & Debugging Notes

8.1 Chatbot Manual Integration Tests

- **Query:** What PPE is required for chemical handling?
 - **Result:** Pass. Correctly retrieved `MSDS Chemical Handling Procedure` using contextual chunking.
- **Query:** Out-of-domain query.
 - **Result:** Pass. LLM safely stated it could not find relevant information, preventing hallucination.

8.2 Debugging Notes

- **BM25 Pre-warming:** Adding a warmup retrieval on startup eliminated a 10-15s cold-start latency.
- **PaddleOCR Versioning:** PaddleOCR 2.9.1 is strictly pinned. Upgrading to 3.x introduced a `pir::ArrayAttribute` bug causing silent OCR extraction failures.

- **Mock Mode:** `PIPELINE_MODE=mock` is essential for running ETL tests without Azure credentials.

9. Project Phases & Next Steps

- **Phase 1 — Foundation Prototype:** Ollama + quantised Llama 3.1 8B, ChromaDB, BGE-m3 embeddings on Hyper-V. Establish ground-truth evaluation metrics.
- **Phase 2 — Retrieval Enhancement:** Implement Hybrid RAG, cross-encoder re-ranking, and Contextual RAG enrichment. Deploy Langfuse monitoring.
- **Phase 3 — Advanced Evaluation:** Implement agentic workflows (LangGraph) and test Azure OpenAI fallback options for complex edge cases.

10. External Libraries & Dependencies

- **Python ML/ETL:** LangChain 0.3.7, ChromaDB 0.5.15, sentence-transformers, pdfplumber, PaddleOCR 2.9.1, MSAL, grpcio, tenacity, pytest.
- **Frontend:** Vue 3, Vite, Pinia, Vue Router, @azure/msal-browser, @stomp/stompjs, DOMPurify, Bootstrap 5.
- **Infrastructure:** K8s, Azure Container Apps, Nginx.

11. Version Control & CI/CD Strategy

To ensure high code quality, parallel development capabilities, and reliable deployments across our microservices architecture, the O'Connors AI Platform implements a robust version control and Continuous Integration/Continuous Deployment (CI/CD) strategy.

11.1 Core Version Control System

The project utilizes Git as the core version control system, with the source code repositories hosted and managed on Bitbucket (GitBucket). This centralized repository management allows the distributed Capstone Project Team to collaborate effectively, track changes securely, and maintain a strict history of all architectural and codebase modifications.

11.2 Repository Structure & Branching Convention

Given the microservices nature of the platform, the version control strategy is designed to isolate service-level changes:

- **Monorepo / Service Isolation:** The project utilizes a structured repository where each microservice (e.g., auth-service, chat-service, etl-service) resides in its own designated directory with independent .gitignore configurations and dependency management files.
- **Feature Branching :** All new developments, bug fixes, or ETL algorithm updates are executed on isolated feature branches. Branches are named according to their corresponding Agile/SCRUM ticket identifiers (e.g., feature/SCRUM-91-resilience-decorator).

- **Pull Request (PR) Gateways:** Direct commits to the main or master branch are strictly prohibited. Developers must submit a Pull Request, which requires peer review and passing automated checks before the code can be merged into the production branch.

11.3 CI/CD Pipeline Automation

Leveraging Bitbucket Pipelines, the platform automates the build and deployment lifecycle to align with the *Essential Eight* compliance and modern DevOps practices:

1. **Automated Testing:** Upon pushing to a feature branch or creating a PR, the pipeline automatically executes unit tests (e.g., pytest for Python services, JUnit for Java services) to catch regressions early.
2. **Containerization:** Once merged into the main branch, the pipeline builds standalone Docker images for the updated microservice.
3. **Registry Push & Deployment:** The newly built Docker image is automatically pushed to the **Azure Container Registry (ACR)** (oconnoraipatform.azurecr.io). Subsequently, the Azure Container Apps environment triggers a zero-downtime rolling update to deploy the latest revision.

11.4 Independent Service Versioning

Because of the microservices architecture, deploying an update to the ETL Service does not require rebuilding or redeploying the User Interface or Chat Service. Additionally, shared internal libraries are versioned independently and published to internal artifact feeds to ensure dependency stability across the platform.

12. References

- Amazon Web Services. (2022). *Serverless Microservices Reference Architecture*.
- Cockcroft, A. (2016). *Adopting microservices at Netflix: Lessons for architectural design*. O'Reilly Media. <https://www.oreilly.com/ideas/microservices-adoption-at-netflix>
- Conway, M. E. (1968). How do committees invent? *Datamation*, 14(4), 28–31.
- Dragoni, N., Giallorenzo, S., Lafuente, A. L., Mazzara, M., Montesi, F., Mustafin, R., & Safina, L. (2017). Microservices: Yesterday, today, and tomorrow. In *Present and ulterior software engineering* (pp. 195–216). Springer. https://doi.org/10.1007/978-3-319-67425-4_12
- Evans, E. (2003). *Domain-driven design: Tackling complexity in the heart of software*. Addison-Wesley.
- Fette, I., & Melnikov, A. (2011). *The WebSocket protocol* (RFC 6455). Internet Engineering Task Force. <https://datatracker.ietf.org/doc/html/rfc6455>
- Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Doctoral dissertation, University of California, Irvine]. <https://roy.gbiv.com/pubs/dissertation/top.htm>
- Fielding, R. T., Nottingham, M., & Reschke, J. (2022). *HTTP semantics* (RFC 9110). Internet Engineering Task Force. <https://datatracker.ietf.org/doc/html/rfc9110>

- Fowler, M., & Lewis, J. (2014, March 25). *Microservices*. martinowler.com. <https://martinfowler.com/articles/microservices.html>
- Google LLC. (2023). *gRPC documentation*. <https://grpc.io/docs/>
- Kniberg, H., & Ivarsson, A. (2012). *Scaling agile @ Spotify*. Spotify. <https://blog.crisp.se/wp-content/uploads/2012/11/SpotifyScaling.pdf>
- Microsoft Azure. (2024). *Azure Container Apps documentation*. <https://learn.microsoft.com/en-us/azure/container-apps/>
- Mozilla. (2024). *WebSocket API*. MDN Web Docs. <https://developer.mozilla.org/en-US/docs/Web/API/WebSocket>
- Netflix Technology Blog. (2015). "Adopting Microservices at Netflix."
- Newman, S. (2021). *Building microservices: Designing fine-grained systems* (2nd ed.). O'Reilly Media.
- Pimentel, V., & Nickerson, B. G. (2012). Communicating and displaying real-time data with WebSocket. *IEEE Internet Computing*, 16(4), 45–53. <https://doi.org/10.1109/MIC.2012.64>
- Rescorla, E. (2018). *The transport layer security (TLS) protocol version 1.3* (RFC 8446). Internet Engineering Task Force. <https://datatracker.ietf.org/doc/html/rfc8446>
- Richardson, C. (2018). *Microservices patterns: With examples in Java*. Manning Publications.
- Taibi, D., & Lenarduzzi, V. (2018). On the definition of microservice bad smells. *IEEE Software*, 35(3), 56–62. <https://doi.org/10.1109/MS.2018.2141031>
- Uber Engineering Blog. (2016). *Introducing Ringpop: Consistent hash ring*. <https://www.uber.com/en-US/blog/ringpop-open-source-nodejs-library/>
- Venkataraman, H., Jagannath, J., & Prasad, N. (2018). Performance analysis of REST and gRPC in microservices. *International Journal of Computer Applications*, 180(40), 1–6.
- Vogels, W. (2006, October 18). *Working backwards*. All Things Distributed. https://www.allthingsdistributed.com/2006/11/working_backwards.html
- Xie, F., & Peng, H. (2020). Research on hybrid search strategy combining dense retrieval and sparse retrieval for question answering. *Proceedings of the 2020 International Conference on Information Technology*, 42–47.
- Reddit r/Rag, "Document markdown and chunking for all RAG", <https://www.reddit.com/r/Rag/comments/1omzhfx/>
- Reddit r/Rag, "Best document format for RAG chatbot", <https://www.reddit.com/r/Rag/comments/1ongnea/>
- GitHub, athina-ai/rag-cookbooks, <https://github.com/athina-ai/rag-cookbooks>
- Reddit r/mlops, "Building a RAG Chatbot for Company", <https://www.reddit.com/r/mlops/comments/1i5vzlf/>
- GitHub, pipeshub-ai, <https://github.com/pipeshub-ai/pipeshub-ai>
- GitHub, Local-LLM-Stack-Documentation, <https://github.com/MuratBuker/Local-LLM-Stack-Documentation>
- Reddit r/LLMDevs, "Chatbot for tons of PDFs", <https://www.reddit.com/r/LLMDevs/comments/1ks8be9/>
- Reddit r/Rag, "Trying to build a RAG chatbot, turned into my worst nightmare", <https://www.reddit.com/r/Rag/comments/1hl9x4i/>
- Reddit r/Rag subreddit, <https://www.reddit.com/r/Rag/>
- Encore.dev, "Best Vector Databases 2026", <https://encore.dev/articles/best-vector-databases>

- Firecrawl, "Best Vector Databases 2026", <https://www.firecrawl.dev/blog/best-vector-databases>
- LiquidMetal AI, "Vector DB Comparison", <https://liquidmetal.ai/casesAndBlogs/vector-comparison/>
- Athenic, "LangChain vs LlamaIndex vs Haystack", <https://getathenic.com/blog/langchain-vs-llamaindex-vs-haystack-rag-frameworks>
- AIMultiple, "RAG Frameworks Benchmarked", <https://aimultiple.com/rag-frameworks>
- Anthropic, "Contextual Retrieval", <https://www.anthropic.com/news/contextual-retrieval>
- Azure OpenAI Pricing, <https://azure.microsoft.com/en-us/pricing/details/cognitive-services/openai-service/>
- Inference.net, "Azure OpenAI Pricing Explained 2026", <https://inference.net/content/azure-openai-pricing-explained>
- Stack Overflow, "Chunking in RAG applications", <https://stackoverflow.blog/2024/12/27/breaking-up-is-hard-to-do-chunking-in-rag-applications/>